I2C bus

serial data line(SDA), serial clock line(SCL)로 bidirectional 2-wire bus

mode	speed	feature
Standard-mode	100kbit/s	bidirectional, serial, 8-bit
Fast-mode	400kbit/s	bidirectional, serial, 8-bit
Fast-mode Plus	1Mbit/s	bidirectional, serial, 8-bit
High-speed mode	3.4Mbit/s	bidirectional, serial, 8-bit
Ultra Fast-mode	5Mbit/s	**uni-bidirectional**, serial, 8-bit

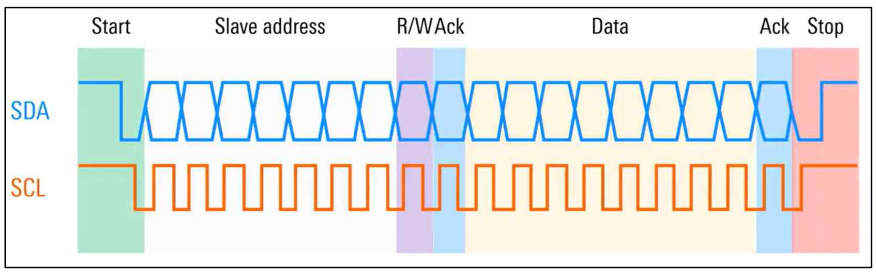
<장점>

clipping과 unclipping을 이용한 수정, address & data transger porotocol은 전부 software-defined, 디버깅쉬움.
각각의 addressing과 enable signal없이 다양한 peripheral 연결 가능
낮은 전력 소비, 고잡음 내성, 넓은 전류원 범위, 넓은 온도 범위 → 휴대용장비, battery-backed system

<활용>

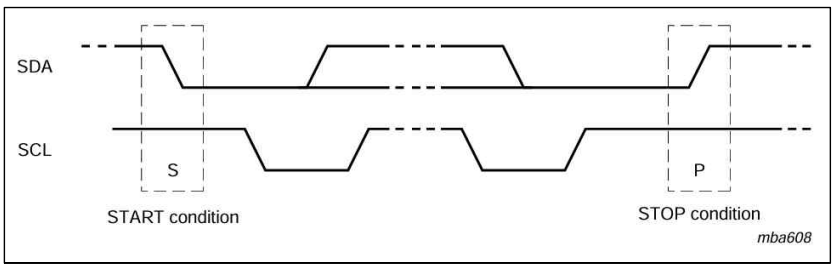
1. CBUS
2. SMBus (System Management Bus)
3. PMBus (Power Management Bus)
4. IPMI (Intelligent Platform Management Interface)
5. ATCA (Advanced Telecom Computing Architecture)
6. DDC (Display Data Channel)

<I2C format>



[frame]

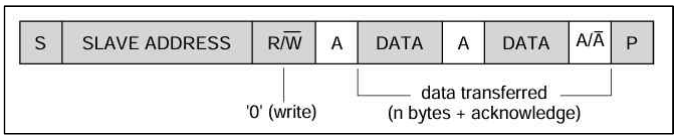
- **START(S or Sr) / STOP(P)**



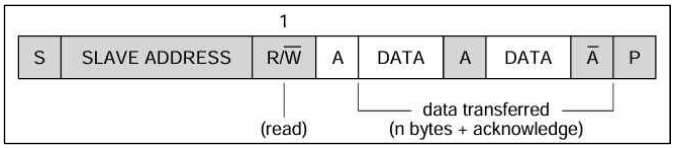
초기 상태 : SDA, SCL → HIGH
SCL이 low일 때만 data 변화 (clock edge x)
+) SCL/SDA = HIGH → STOP

- **slave address & R/W bit / data**

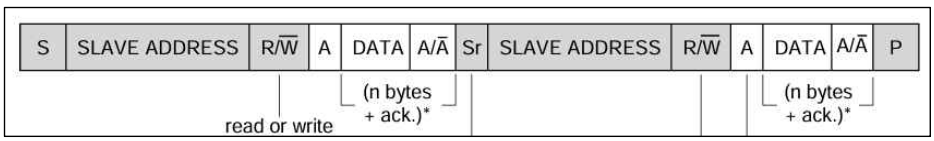
: 8-bit long (slave address(7bit) + R/W(1bit) or data (8bit))
각 device는 unique address를 가지고 있다.
ACK bit에 의해 follow, MSB먼저
R/W = 0 (**write** data to slave), R/W = 1 (**read** data from slave)



1. master가 slave에게 7-bit address를 줄 때



2. transfer가 일어났을 때



3.combined 됐을 때

- **ACK (Acknowledge) & NACK (negative acknowledge)**

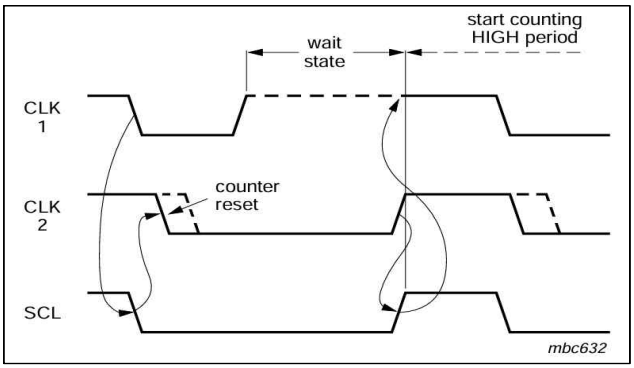
: transmitter에게 byte가 성공적으로 보내졌다고 알리는 신호

ACK는 모든 byte마다 일어남.

+) **ACK** : SDA 9th clock pulse에 LOW로 유지 , **NACK** : SDA가 9th clock pulse에 HIGH로 유지

- **Clock Synchronization**

: 2개 이상의 master가 같은 시간에 method를 결정할 때 → **wired-AND connection**이용



C1	C2	SCL
0	0	0
0	1	0
1	0	0
1	1	1

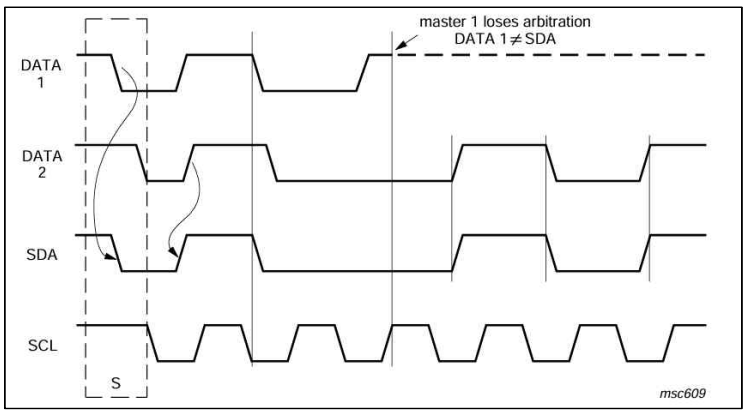
[wired-AND]

SCL LOW = 가장 느린 master기준

SCL HIGH = 가장 빠른 master기준

- **Arbitration**

: 두 개 이상의 마스터가 START 조건을 거의 동시에 발생시키면 누가 버스를 사용할지 결정 (slave 관여x)

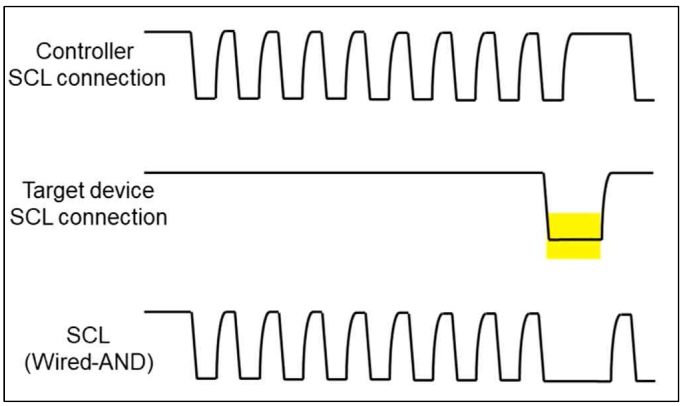


- Bit-by-Bit : **매 bit마다 SCL이 HIGH일 때 master는 자기가 보낸 SDA값과 실제 SDA값 비교**

ex) 어떤 master가 HIGH를 보내려 했는데 bus에서 LOW가 감지되면 해당 master는 arbitration에서 lose (LOW가 HIGH보다 우선권을 가짐.) → open-drain 구조

- **Clock Stretching**

: ACK이후 **SCL line을 LOW로 holding**하는 것.

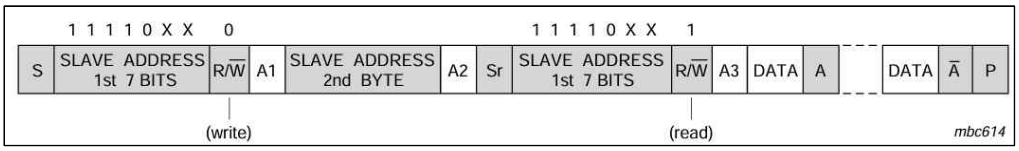


SCL이 stretched된다.

1. ACK 이후, 다음 바이트를 준비하지 못했을 때
2. 수신한 바이트를 내부에 저장/처리할 시간이 필요할 때
3. 송신할 데이터를 아직 준비하지 못했을 때
4. (느린 MCU의 경우) 비트 단위로 클럭을 늦춰야 할 때

- **10-bit addressing**

: 주소 개수 확장을 위함 (standard, fast, fast +)



'11110xx' (상위 2bit) + 하위 8bit = 10bit

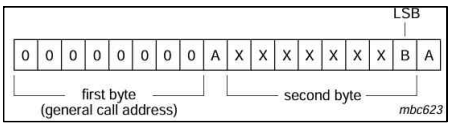
- **Reserved address**

: 특정 목적을 위한 주소

'0000 xxxx' '1111 xxxx'

▷ **General call address : '0000 0000'**

→모든 device를 동시에 주소 지정하는 주소 (반드시 응답x, ACK로 확인)



second byte : 명령 코드 + B비트 (**B=0** → 특정 기능 코드, **B=1** → 자신이 누구인지 system에 알림)

ex) Software reset : general call이후 '0000 0110' 전송 시 발생

▷ **START Byte : '0000 0001'**

→ MCU는 polling과 interrupt방식을 사용하는데 polling을 사용할 시 빠른 device에 비해 느리기 때문에 **속도차이**로 인해 START Byte를 이용해 더 긴 시작 절차를 가짐.

S → Start Byte → ACK → Sr (*ACK는 형식을 맞추기 위함.)

▷ **Device ID (optional)** : 디버깅에 유용, read-only(24bit)

manufacturer(12bit) + part identification (9bit) + revision(3bit)

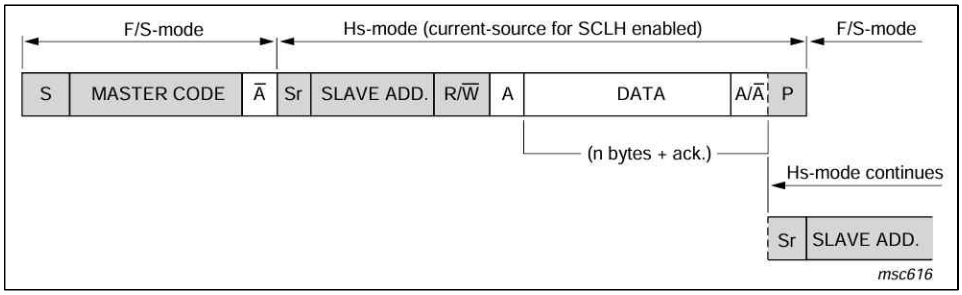
- **Bus clear** : SCL or SDA가 stuck low될 때 사용

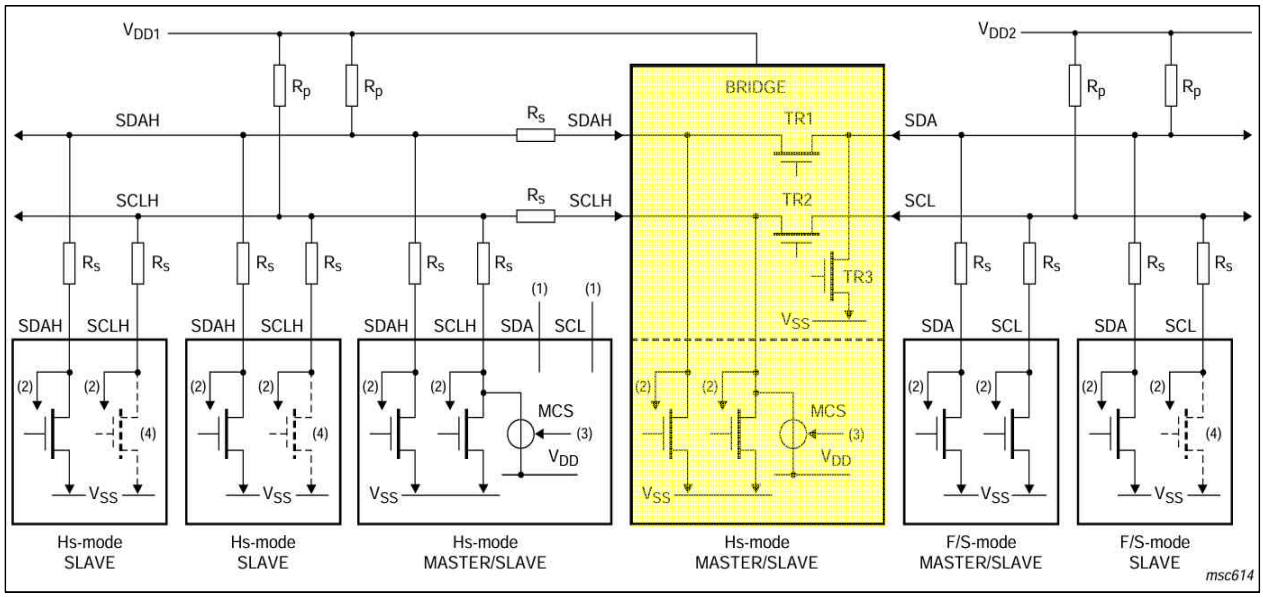
ex) H/W reset이용, device의 전원을 껐다 켜 내부 POR(Power-On Reset)회로 동작

<Hs-mode>

: 기존 구조와 동일 (arbitration & clock synchronization 기증 제외 → 높은 속도를 목표)

- SDAH (open-drain output buffer)
- SCLH (open-drain pull-down & current-source pull-up)

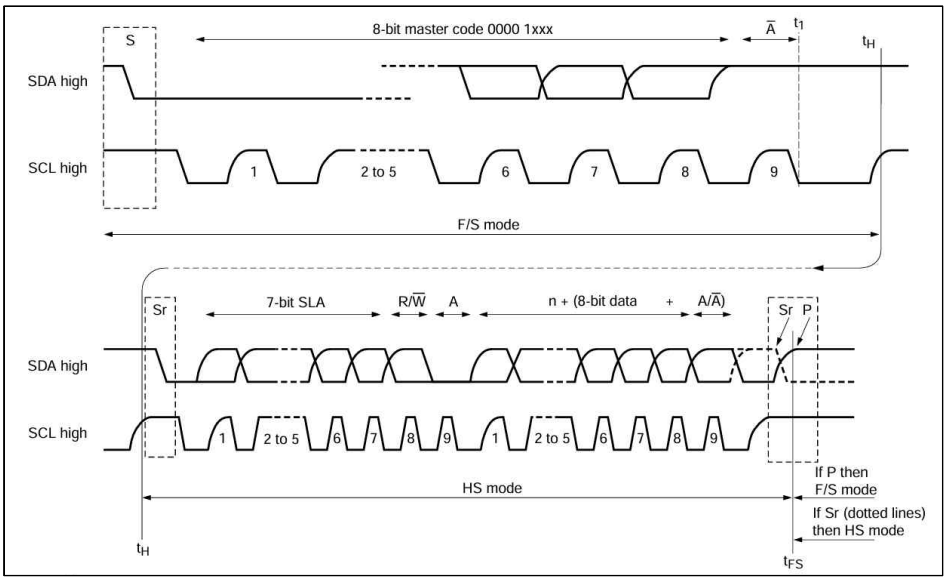




*) Rs : 고전압 sike suppression, Rp : Pull-up 저항

▶ 모든 mode가 함께 존재 → interconnection bridge 사용 (서로 다른 bit-rate로 통신)

▶ F/S mode : SDAH ↔ SDA, SCLH ↔ SCL 연결 / Hs-mode : 두 bus 분리



1. Hs-mode 전 F/S mode

: S + master code ('0000 1xxx') + NACK

2. t1 ~ tH

: TR1= open → SDAH, SDA line분리, TR3 = closed → SDA = VSS (pull-down)

3. tH

: SCLH, SCL = HIGH → TR2 = open → SCLH,SCL 분리

4. Hs-mode 전송

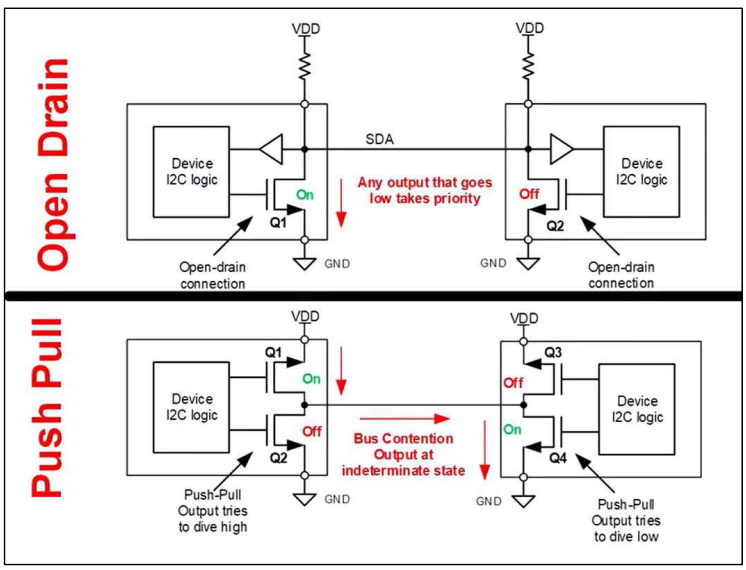
5. Sr 이후 (SCL =HIGH, SDA=LOW 유지)

: ACK/NACK 이후 active master는 current-source pull-up 비활성화 → 모든 device가 SCLH를 해제하고 HIGH도달 → MCS 다시 활성화 (*SCLH상승 시간의 마지막 구간을 빠르게 만듦.)

<Ultra Fast-mode>

: 2-wire push-pull구조 (USDA,USCL) → pull-up 저항 없음

S → **slave address** → W → NACK → P (write-only, ACK 필요x)



push-pull → multi-master, arbitration, clock synchronization 없음.

if) master 2개 필요 시 분리된 MUX필요.

*) open-drain구조 : LOW → device가 직접, HIGH → Rp

+) Device ID는 UFm에선 지원 x